TERMINI · AI BUILDER CUP · MANUFACTURING — INTELLIGENT OPERATIONS & INDUSTRIAL EFFICIENCY

Termini AI-Infra

Turning bare, on-premise hardware into a **self-operating AI fleet** — a cloud control plane bootstraps it, a **governed AI operator** runs it, and **your data never leaves**.

“Your hardware. Your data. Our operations.”

Day-0 bootstrap | Day-2 self-healing | Fail-closed governance

AI is stuck at the on-prem gate

Enterprises holding governance-sensitive data — customer records, source code, national-core IP — **cannot ship that data to a SaaS cloud** to get AI. They are stuck at three points no vendor has productized into one thing.

1 Day-0 — turn it on

Standing up an AI-ready cluster (hypervisor, GPU serving, networking, agent VMs) today requires a specialist. A small or regulated business has **no SRE bench** to do it.

2 Day-2 — keep it alive

Once it runs, someone must watch disks, memory and GPU health and fix failures. On-prem means **you are the operations team**.

3 Governance — prove it

“Your data never leaves” has to be an **enforceable property**, not a slogan in the sales deck.

Ops can be delegated. Data doesn’t have to follow.

A Datadog proved delegation

Best-in-class day-2 observability plus an agentic surface. But it is **SaaS-architected** — telemetry, traces and logs ship to *its* cloud, with no self-hosted / air-gapped AI offering. And it assumes the infrastructure **already exists**.

B Samsung × Mistral proved demand

The Sept 2026 partnership keeps AI on-prem for data sovereignty. But it is **bespoke, equity-backed**, mega-corp — and covers **models & platform, not day-2 operations**. Small businesses can’t replicate it.

The **missing middle**: nobody productizes the **bootstrap** + the **day-2 operator** + the **governance boundary** in one thing.

“Datadog proved ops can be delegated — but it does it by moving your telemetry to their cloud. We do the same job for people **who can’t send anything off-prem**.”

Bootstrap the hardware. Operate it with a governed AI.

A cloud control plane bootstraps the hardware on day-0, an AI operator remediates it on day-2, and a fail-closed governance boundary makes data residency **checkable — not asserted**.

Telemetry → **Threshold rule** → **AI decides (Gemma) — allowlist-constrained** → **Audited LXD execution**

+ We build two things

The **governance boundary** and the **governed AI-operator control loop** — the moat.

= We adopt the rest

The substrate (LXD + Ubuntu), the connectivity (underlay + SSH), the model (llama.cpp + GGUF). Model choice is a **config knob** — the product is the ops + governance layer.

Same delegation. Opposite data flow.

vs Datadog (infra ops)

- Datadog **exfiltrates** telemetry to its SaaS; Termini runs a **self-hosted remediation loop** — intent extraction on the customer’s own hardware.
- Datadog has **no self-hosted / air-gapped AI**; Termini serves an entire market that can’t send anything off-prem.
- Datadog assumes the infra **already exists**; Termini owns day-0 bootstrap too.
- Only **C0 hardware/OS telemetry** ever leaves Termini’s boundary.

vs Samsung × Mistral (on-site AI)

- Bespoke, equity-backed, **mega-corp** engagement — not replicable by an SMB.
- Covers **models and platform**, not maintenance and operations.
- Termini **productizes the long tail**: open-weight self-hosted models **plus** the bootstrap and the governed operator.
- Samsung has expert engineers; **SMBs do not** — that’s the wedge.

Three proof points

1 Bare metal → AI-ready fleet

A node booted from the **pre-baked image** registers to the controller in **150 ms** — measured, against the 5-second budget — and joins the cluster with its role assigned. **No SRE, no manual config, and nothing installed at boot.**

2 NL intent, guardrail-enforced

Type *“deploy 3 worker agents for coding — plus a fast LLM.”* An on-prem model extracts a structured allocation; the **deterministic guardrail clamps** the impossible into a **validated, clamped plan** — and the fast-LLM reference resolves to the always-on Node C inference service, not a VM.

3 Self-healing with a boundary

OOM a VM live. Telemetry crosses a threshold, the operator **detects → decides → executes → audits** — while a C2 field is **rejected by the governance gate before any byte is stored**.

Q1 What operations scenarios do we showcase, and how do we simulate them?

RESOLVED

Two real failure scenarios, each remediated autonomously — A: out-of-memory and B: disk degradation — injected with off-the-shelf tools, in both cases diagnosed and remediated by the model.

A Out-of-memory

stress-ng burns RAM, starves neighbor agents → operator detects, **restarts** the offending workload, restores the fleet.

B Disk degradation

Injected **I/O error** (write latency climb / read-only mount) → operator **flags the drive, stops the heaviest workload, alerts** the user.

Contested: beating the kernel OOM-killer is best-effort. Resolved with **1000 ms telemetry** + a rule that fires **deterministically** (the trigger is never a model) + a **deterministic chaos ramp**, with the decision made by **Gemma on an L4** — seconds, not the 50–60 s measured on CPU — and a **mandatory deterministic fallback**. We demonstrate the **detect → decide → execute → audit** loop end to end.

HOW THE DEMO PROVES IT — dashboard shows RAM crossing 95% → an **instance.restart** acks **succeeded** → the workload recovers, traceable end-to-end in the audit log.

Q2 What cloud system ties the nodes together — LXD or OpenStack — and how do they share a network?

RESOLVED

LXD (5.21/stable) as a 3-member cluster. OpenStack rejected — hours-to-days of setup for problems this demo doesn't have.

Inference path — no overlay

Each node keeps a **local NAT bridge** on its own subnet; the **inference leg rides the physical underlay by real IP**. The on-prem inference service (**llama.cpp**) on Node C is reachable at **http://10.10.0.13:8080**. Agent-workload addressing uses a **VXLAN overlay** (**10.99.0.0/24**); the latency-critical inference leg stays on the underlay.

No tunnel on the inference leg

On the inference leg, a single static route either works or it doesn't — **no encapsulation**. Guest MTU clamped to the underlay (**1460**) to avoid PMTU black holes.

Contested: overlay tunnels (OVN/Fan/VXLAN) silently drop packets on MTU mismatch and add per-call latency. The latency-critical inference leg avoids the overlay entirely.

HOW THE DEMO PROVES IT — from an agent VM on Node A, **curl http://10.10.0.13:8080/v1/models** returns the model list over standard HTTP; watch **traceroute** and streamed tokens with no tunnel on the inference leg.

Q3 How do we smartly allocate agent-VM resources — can we infer them from natural language?

RESOLVED

Yes. Plain English → intent extraction (on-prem, JSON via json_object response format) → deterministic validator. The model recommends; the validator decides.

✓ Hard caps

- **vcpus ≤ 128**, **ram ≤ 512 GB**, **count ≤ 64**
- Per-node ceilings with **host reserve**
- Per-type ranges (agent_vm: vcpus 1–8, ram 1–16 GB)

✕ All-or-nothing

- Numbers are **clamped** to real capacity
- Structural violations **rejected** outright
- A rejected class creates **nothing**

Contested: a prompt like “super fast agent” yielding **vcpus:128, ram:512** that LXD can’t honor. Resolved with **deterministic hard guardrails, not model politeness**.

HOW THE DEMO PROVES IT — type **“128 vCPU, 512 GB RAM worker”** → the plan shows **clamped values** within the host’s real capacity, with per-field **from → to** clamping visible in the audit record.

Q4 How do we serve the pre-baked agentic OS — what's pre-installed, and how do we hand it over AI-ready (SSH-able)?

RESOLVED

Immutable pre-baked images — nothing downloads at boot. cloud-init only activates & registers: 150 ms measured against a 5 s budget. Access is browser-first.

◈ Access without passwords

- Keypair generated **in the browser** — only the public half is uploaded
- Controller's **SSH CA** signs a **short-lived cert** (≤8 h, principal-bound)
- One-click **web terminal** + an **ssh** string, revocable — **no passwords**

▣ Data boundary (C0/C1/C2)

- Only **C0 hardware/OS telemetry** + **C1 transformed IDs** egress
- **C2** (logs, payloads) **never leaves**
- Controller-side gate **fails closed** — a C2 batch is rejected whole
- External model calls scoped to **C0 only**

HOW THE DEMO PROVES IT — power on → **registered in 150 ms** with zero installs; open the terminal for a **passwordless shell**; inject a C2 field and watch the gate **reject the batch**; pull the operator audit and see every action traceable by its **call_id**.

10 LIVE DEMO

What you are about to see

A live cluster on cloud VMs standing in for on-premise bare hardware. Four moments, in order:

1 **Node powers on → registered in 150 ms**
Pre-baked image activates, joins the cluster, role assigned. No package installs.

2 **Plain English → validated plan**
“Deploy 3 worker agents for coding, plus a fast LLM.” Spec extracted on-prem — the fast-LLM reference resolves to the Node C inference service — clamped, validated.

3 **OOM a VM — live**
The operator detects, decides, executes the fix, and writes the audit trail.

4 **Zero C2 data leaves**
Watch the governance gate reject a C2 batch fail-closed.

▸ Narration takes over now — over to the live cluster.

11 RESULTS

What we just demonstrated

150 ms

Node registration from the pre-baked image — **zero installs at boot**

clamped

Natural-language intent → **valid allocation**, from→to clamping visible

audited

Gemma-chosen OOM remediation, **end-to-end auditable**

0

C2 egress — the boundary held on every call

1

Fix executed autonomously (restart) from a threshold rule

3

Proof points shown live, not claimed

The AI runs the operations — live, on Google Cloud

No playbook lookup chooses the fix. The threshold detects the fault; **Gemma** — an open-weights Google model — reasons about it and selects the remediation, which then executes against the cluster and lands in the audit trail.

1 Gemma 3 4B decides

Telemetry in, action out — the model diagnoses the fault and picks the fix. Served by **vLLM on Cloud Run GPU (1× NVIDIA L4)** in **asia-southeast1**.

2 Constrained, never trusted

A **GBNF grammar** makes an off-allowlist command **unencodable** — safety is decoder-enforced, not a fail-open validator — plus param guards, a confidence gate, and a **mandatory deterministic fallback**.

3 Deployed, not mocked

Cloud Run serves the model · **Firebase Hosting** the dashboard · **GCE** the controller · **GCS** the weights. A judge can open it and watch it run.

autonomous

Faults remediated with **no human in the loop**

0

C2 egress — the model only ever sees C0 telemetry

open

Weights you can **self-host** — no closed-model lock-in

What’s next: from prototype to production

- **Learned baselines**
  Netdata edge-ML anomaly detection replaces static thresholds.
- **HA shared storage**
  MicroCloud / MicroCeph for shared storage, multi-tenant isolation, no migration copy.
- **Model-agnostic serving**
  Swap DeepSeek distills for any open-weight model — the product is ops + governance, not the weights.
- **Scale & retention**
  VictoriaMetrics for persistence; k3s when agent scheduling moves to Kubernetes.
- **Real bare-metal onboarding**
  MAAS + iPXE netboot to onboard customer hardware end-to-end.
- **Compliance alignment**
  GDPR, EU AI Act, NIS2, DORA posture mapped to the governance boundary.

TERMINI · AI-INFRA

Your hardware.
Your data.
Our operations.

Bootstrap day-0. Self-heal day-2. Prove the data stays.

Questions

Live demo available